



赞同 54

分享

扩散模型解读 (一): DiT 详细解读

 科技猛兽  
清华大学 自动化系硕士

54 人赞同了该文章

本系列已授权极市平台，未经允许不得二次转载，如有需要请私信作者。

专栏目录

科技猛兽：扩散模型超详细解读 (目录)
39 赞同 · 1 评论 文章



本文目录

- 1 DiT: Transformer 构建扩散模型
(来自 UCB, NYU, William Peebles, Saining Xie 重量级团队)
 - 1 DiT 论文解读
 - 1.1 把 Transformer 引入 Diffusion Models
 - 1.2 Diffusion Models 简介
 - 1.3 DiT 架构介绍
 - 1.4 DiT 训练策略
 - 1.5 DiT 的探索过程
 - 1.6 DiT 实验结果

太长不看版

本文探索了一类新的基于 Transformer 的扩散模型 Diffusion Transformers (DiTs)。本文训练 latent diffusion models 时，使用 Transformer 架构替换常用于 latent patches 上。

 赞同 54   2 条评论  分享  喜欢  收藏

256×256 的测试中优于所有先前的扩散模型，实现了 2.27 的 FID 值。

本文做了什么工作

1. 探索了一类新的基于 Transformer 的 Diffusion Model，称为 Diffusion Transformers (DiTs)。
2. 研究了 DiT 对于模型复杂度 (GFLOPs) 和样本质量 (FID) 的缩放性。
3. 证明了通过使用 Latent Diffusion Models (LDMs)^[1] 框架，Diffusion Model 中的 U-Net 架构可以被 Transformer 替换。

1 DiT: Transformer 构建扩散模型

论文名称: Scalable Diffusion Models with Transformers (ICCV 2023, Oral)

论文地址:

<https://arxiv.org/pdf/2212.09748.pdf>
arxiv.org/pdf/2212.09748.pdf

论文主页:

<https://www.wpeebles.com/DiT.html>
www.wpeebles.com/DiT.html

- 1 DiT 论文解读:

1.1 把 Transformer 引入 Diffusion Models

机器学习正经历着 Transformer 架构带来的复兴: NLP, CV 等许多领域正在被 Transformer 模型覆盖。尽管 Transformer 在 Autoregressive Model 中得到广泛应用^{[2][3][4][5]}, 但是这种架构在生成式模型中较少采用。比如, 作为图像领域生成模型的经典方法, Diffusion Models^{[6][7]}却一直使用基于卷积的 U-Net 架构作为骨干网络。

Diffusion Models 的开创性工作 DDPM^[8]首次引入了基于 U-Net 骨干网络的扩散模型。U-Net 继承自 PixelCNN++^{[9][10]}, 变化很少。与标准 U-Net^[11]相比, 额外的空间 Self-Attention 块 (Transformer 中必不可少的组件) 以较低分辨率穿插。^[12]这个工作探索了 U-Net 的几种架构选择, 例如自适应归一化层 (Adaptive Normalization Layer^[13]) 为卷积层注入条件信息和通道计数。然而, DDPM 里面 U-Net 的高级设计在很大程度上都保持不变。

本文的目的是探索 Diffusion Models 架构选择的重要性, 并为未来生成式模型的研究提供基线。本文的结论表明 U-Net 架构设计对 Diffusion Models 的性能并不重要, 并且它们可以很容易地替换为 Transformers。

本文证明了 Diffusion Models 也可以受益于 Transformer 架构, 受益于其训练方案, 受益于其可扩展性, 受益于其鲁棒性和效率等等。标准化架构还将为跨域研究开辟了新的可能性。

1.2 Diffusion Models 简介

DDPM

高斯扩散模型假设有一个前向的加噪过程 (Forward Noising Process), 在这个过程中逐渐将噪声应用于真实数据:

$$x_0 : q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

(1)

其中, 常数 $\bar{\alpha}_t$ 是超参数。通过使用 reparameterization trick

式中, $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 。

Reparameterization trick^[14]就是把一个随机变量改写为噪声变量的确定性函数, 这样就可以通过梯度下降来优化这个非随机变量了。比如有一个 $\mathbf{x} \sim \mathcal{N}(\mathbf{x}; \mu, \sigma^2)$, 那么 $\mathbf{x}$ 就可以被重写为:

$$\mathbf{x} = \mu + \sigma \epsilon, \text{ with } \epsilon \in \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{I})$$

所以有2式成立。

Diffusion Model 就是去学习一个反向过程: 给定当前 step 的图片 $\mathbf{x}_t$, 通过网络参数 θ 来预测上一个 step 的图片 $\mathbf{x}_{t-1}$ 。

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mu_{\theta}(\mathbf{x}_t), \Sigma_{\theta}(\mathbf{x}_t)) \quad (3)$$

这个优化的目标函数比较复杂, 最后通过 variational lower bound 方法得到的结论是优化下式 (此处详细推导可以参考开创性工作 DDPM^[8]):

$$\mathcal{L}(\theta) = -p(\mathbf{x}_0|\mathbf{x}_1) + \sum_t \mathcal{D}_{KL}(q^*(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)) \quad (4)$$

由于 q^* 和 p_{θ} 都是高斯的, 因此 $\mathcal{D}_{KL}$ 可以使用两个分布的均值和协方差进行评估。

扩散模型在训练的时候使用预测噪声 $\epsilon_{\theta}(\mathbf{x}_t)$ 和 GT 值噪声进行训练

$\epsilon_t: \mathcal{L}_{simple}(\theta) = ||\epsilon_{\theta}(\mathbf{x}_t) - \epsilon_t||_2^2$ 。作者使用 $\mathcal{L}_{simple}$ 来训练 ϵ_{θ} , 使用 $\mathcal{L}$ 来训练 Σ_{θ} 。一旦扩散模型 p_{θ} 训练好了以后, 就可以采样新的图片 $\mathbf{x}_{t_{max}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, 然后通过 Reparameterization trick^[14]一步一步采样 $\mathbf{x}_{t-1} \sim p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 。

Latent diffusion models

Latent diffusion models 直接在高分辨率像素空间中训练 Diffusion Model 会导致巨大的计算量。LDM^[1]通过两阶段方法解决这个问题:

1. 学习一个 AutoEncoder, 用学习过的 AutoEncoder E 将图像压缩为更小的空间表征。
2. 在 $\mathbf{z} = E(\mathbf{x})$ 而非原图 $\mathbf{x}$ 上训练一个扩散模型, 这个过程中 E 被冻结。
3. 在生成新图片时, 从扩散模型中采样 $\mathbf{z}$, 再最后经过学习过的解码器解码为图像 $\mathbf{x} = D(\mathbf{z})$ 。

1.3 DiT 架构介绍

1.3.1 Patchify 过程

对于 $256 \times 256 \times 3$ 的图片, $\mathbf{z}$ 的维度是 $32 \times 32 \times 4$ 。DiT 的第1步和 ViT 一样, 都是把图片 Patchify, 并经过 Linear Embedding, 最终变为 T 个 d 维的 tokens。在 Patchify 之后, 作者将标准的基于 ViT 频率的位置编码 (sine-cosine 版本) 应用到所有的输入 tokens 上面。

token T 的数量由 Patch 的大小 p 决定。如下图1所示, Patch 的大小 p 和 token 的数量 T 之间满足 $T = (I/p)^2$ 的关系。当 Patch 的大小 p 越小时, token 的数量 T 越大。减半 p 将会使 T 变为4倍, 使得计算量也变为4倍。尽管 p 对 GFLOPs 有显著的影响, 但参数量没有很实质的影响。

作者在 DiT design space 里面使用 $p = 2, 4, 8$ 。

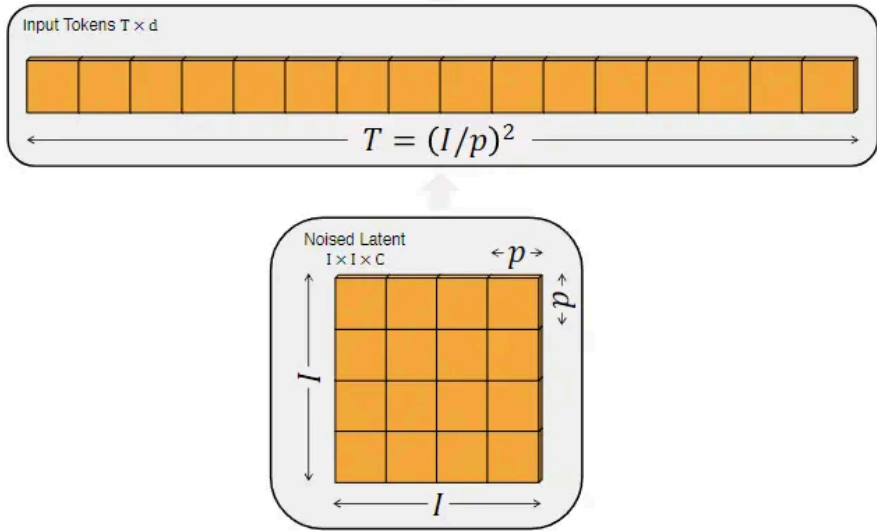


图1: 图片的 Patchify 操作。当 Patch 的大小 p 越小时, token 的数量 T 越大

1.3.2 DiT Block 设计

在 Patchify 之后, 输入的 tokens 开始进入一系列 Transformer Block 中。除了噪声图像输入之外, Diffusion Model 有时会处理额外的条件信息, 比如噪声时间步长 t , 类标签 c , 自然语言。

作者探索了4种不同类型的 Transformer Block, 以不同的方式处理条件输入。这些设计都对标准 ViT Block 进行了微小的修改, 所有 Block 的设计如下图2所示。

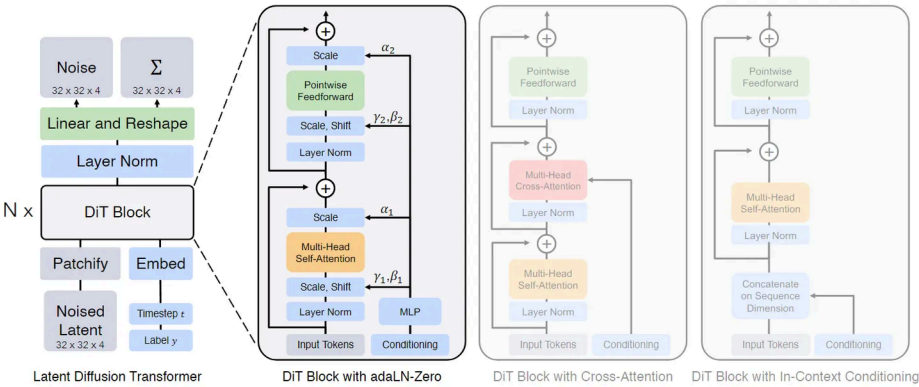


图2: Diffusion Transformer (DiT) 架构

• In-Context Conditioning

像这样的带条件输入的情况, In-context conditioning 的做法只需要将时间步长 t , 类标签 c 作为2个额外的 token 附加到输入的序列中。作者将它们视为与图像的 token 没有任何区别。这就有点类似于 ViT 的 [CLS] token。这就允许 DiT 使用标准的 ViT Block 而不用任何修改。经过了最后一个 Block 之后, 删除条件 token 就行。这种方式带来的额外 GFLOPs 微不足道。

• Cross-Attention Block

Cross-attention block 的做法是将 t 和 c 的 Embedding 连接成一个长度为2的 Sequence, 且与 image token 序列分开。这种方法给 Transformer Block 添加一个 Cross-Attention 块。这个操作带来的额外 GFLOPs 开销是大约 15%。

• Adaptive Layer Norm (adaLN) Block

Adaptive Layer Norm (adaLN) Block 遵循 GAN^[15]中的自适应层归一化, 作者想探索这个在扩散模型里面好不好用。本文作者没有直接学习缩放和移位参

• adaLN-Zero Block

有工作^[16]发现在有监督学习中，对每个 Block 的第一个 Batch Norm 操作的缩放因子进行 Zero-Initialization 可以加速其大规模训练。基于 U-Net 的扩散模型使用类似的初始化策略，对每个 Block 的第一个卷积进行 Zero-Initialization。本文作者作了一些改进：除了回归计算缩放和移位参数 γ 和 β 之外，还回归缩放系数 α 。

作者初始化 MLP 使其输出的缩放系数 α 全部为0，这样一来，DiT Block 就初始化为了 Identity Function。adaLN-Zero Block 带来的额外的 GFLOPs 可以忽略不计。

作者将以上几种方法 In-Context Conditioning, Cross-Attention Block, Adaptive Layer Norm (adaLN) Block, adaLN-Zero Block 的做法列入 DiT 的设计空间中。

1.3.3 模型尺寸

遵循 ViT 的做法，作者在缩放 DiT 时也是从下面几个维度进行考虑：深度 N ，hidden dimension d ，head 数量。作者设计出4种不同尺寸的 DiT 模型 DiT-S, DiT-B, DiT-L 和 DiT-XL，从 0.3 到 118.6 GFLOPs，详细信息如下图3所示。

Model	Layers N	Hidden size d	Heads	Gflops ($l=32, p=4$)
DiT-S	12	384	6	1.4
DiT-B	12	768	12	5.6
DiT-L	24	1024	16	19.7
DiT-XL	28	1152	16	29.1

图3: DiT 模型的详细配置

作者将以上几种配置列入了 DiT 的设计空间中。

1.3.4 Transformer Decoder

在最后一个 DiT Block 之后，需要将 image tokens 的序列解码为输出噪声以及对角的协方差矩阵的预测结果。

而且，这两个输出的形状都与原始的空间输入一致。作者在这个环节使用标准的线性解码器，将每个 token 线性解码为 $p \times p \times 2C$ 的张量，其中 C 是空间输入中到 DiT 的通道数。最后将解码的 tokens 重新排列到其原始空间布局中，得到预测的噪声和协方差。

最终，完整 DiT 的设计空间是 Patch Size、DiT Block 的架构和模型大小。

1.4 DiT 训练策略

1.4.1 训练配方

作者在 ImageNet 数据集上训练了 class-conditional latent DiT 模型，标准的实验设置。

分辨率设为 256×256 和 512×512 。

对最后的 Linear Layer 使用 Zero Initialization，其他使用来自 ViT 的标准权重初始化方案。

优化器使用 AdamW，使用固定学习率 1×10^{-4} ，Batch Size 使用 256，不使用 weight decay。

数据增强技术只使用 horizontal flips。

作者发现 learning rate warmup 和 regularization，对训练

训练超参数基本都来自 ADM，不调学习率, decay/warm-up schedules, Adam 参数以及 weight decay。

1.4.2 扩散模型配置

对于 DiT 的其他组件，作者使用 Stable Diffusion 的预训练好的 Variational AutoEncoder (VAE) 模型。VAE Encoder 下采样率是 8：给定 $256 \times 256 \times 3$ 的输入图片 x ，得到的编码结果 z 尺寸为 $32 \times 32 \times 4$ 。

从扩散模型中 Sample 出一个新的 latent 结果之后，作者使用 VAE decoder 将其解码回 pixel： $x = D(z)$ 。

作者保留了 ADM 中使用的超参数。

1.5 DiT 的探索过程

1.5.1 DiT 架构设计

作者首先探索的是不同 Conditioning 策略的对比。对于一个 DiT-XL/2 模型，其计算复杂度分别是：in-context (119.4 Gflops), cross-attention (137.6 Gflops), adaptive layer norm (adaLN, 118.6 Gflops), adaLN-zero (118.6 Gflops)。实验结果如下图4所示。

adaLN-Zero 的 Block 架构设计取得了最低的 FID 结果，同时在计算量上也是最高效的。在 400K 训练迭代中，adaLN-Zero Block 架构得到的 FID 几乎是 In-Context 的一半，表明 Condition 策略会严重影响模型的质量。

初始化同样也重要：adaLN-Zero Block 架构在初始化时相当于恒等映射，其性能也大大优于 adaLN Block 架构。

因此，在后续实验中，DiT 将一直使用 adaLN-Zero Block 架构。

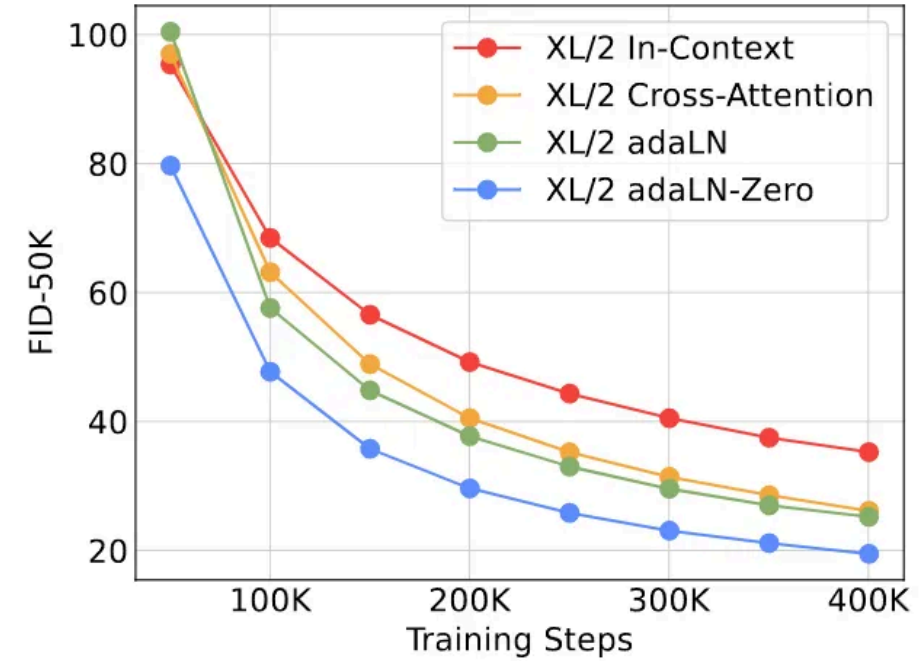


图4：不同 Conditioning 策略对比

1.5.2 缩放模型尺寸和 Patch Size

作者训练了12个 DiT 模型 (尺寸为 S, B, L, XL, Patch Size 为 8,4,2)。下图是不同 DiT 模型的尺寸和 FID-50K 性能。如下图5所示是不同大小 DiT 模型的 GFLOPs 以及在 400K 训练迭代中的 FID 值。可以发现，在增加模型大小或者减小 Batch Size 时可以显著

知乎 首发于 AIGC 日积月累

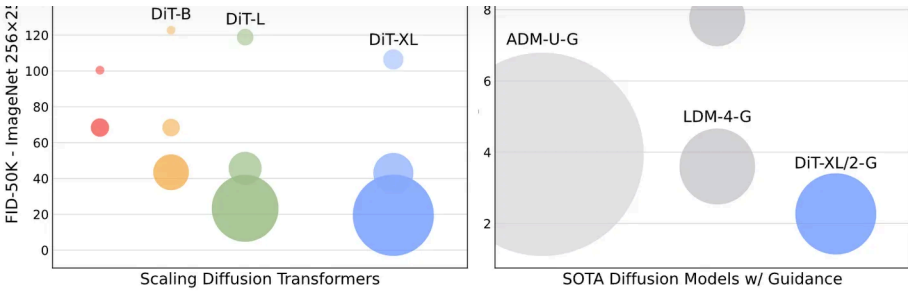


图5: 不同尺寸 DiT 模型的 GFLOPs 以及它们在 400K 训练迭代中的 FID

下图6上方是 Patch Size 不变, 增加模型规模时 FID 的变化。当模型变深变宽时, FID 会下降。
下方是模型规模不变, 减小 Patch Size 时 FID 的变化。当 Patch Size 下降时, FID 出现显著改善。

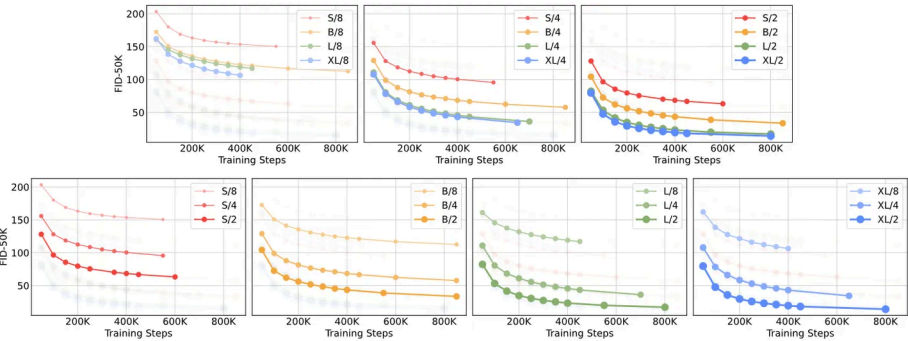


图5: 缩放 DiT 模型可以改善训练各个阶段的 FID

1.5.3 GFLOPs 对性能很重要

上图5的结果表明, 参数量并不能唯一确定 DiT 模型的质量。当 Patch Size 减小时, 参数量仅仅是略有下降, 只有 GFLOPs 明显增加。这些结果都表明了缩放模型的 GFLOPs 才是性能提升的关键。为了印证这一点, 作者在下图6中绘制了不同 GFLOPs 模型在 400K 训练步骤时候的 FID-50K 结果。这些结果表明, 当不同 DiT 模型的总 GFLOPs 相似时, 它们的 FID 值也相似, 比如 DiT-S/2 和 DiT-B/4。

作者还发现 DiT 模型的 GFLOPs 和 FID-50K 之间存在很强的负相关关系。

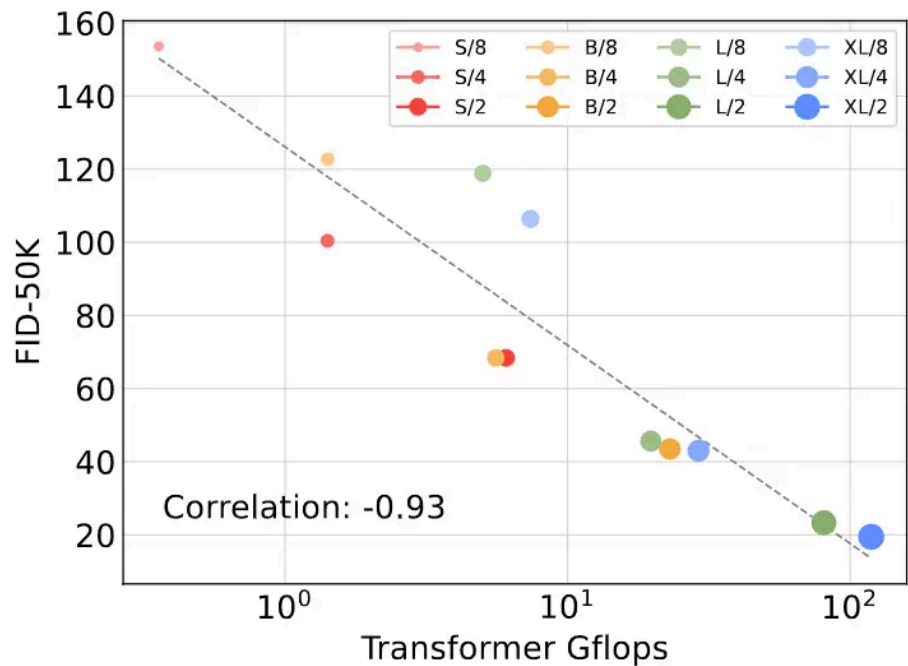


图6: GFLOPs 与 FID 密切相关

1.5.4 大模型更加计算高效

2倍。

作者发现，与更少训练步骤的大的 DiT 模型相比，小的 DiT 模型，即使训练时间更长，最终也会计算效率变低。而且，一样大小的模型，只是 Patch Size 不同时，即使训练的计算资源相当，表现也不同。比如在 10^{10} GFLOPs 时，XL/4 不如 XL/2。

总结起来就是：大的模型计算效率更高，小 Patch Size 的模型计算效率更高。

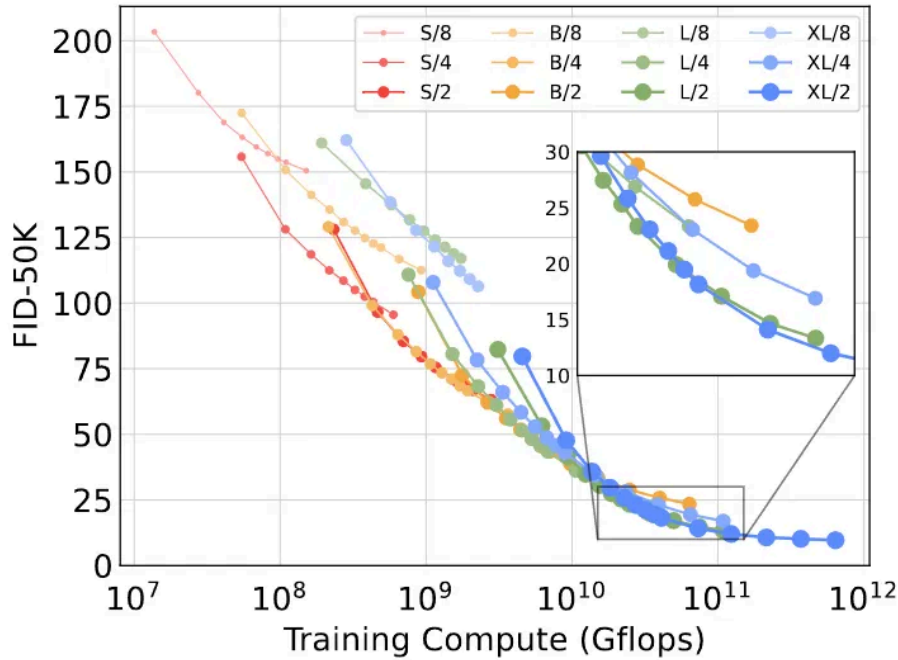


图7：大模型更加计算高效

1.5.5 缩放结果可视化

作者使用相同的起始噪声 $x_{t_{max}}$ ，采样噪声和类别标签从12个 DiT 模型中分别采样在 400K training steps 的生成结果，如下图8所示。这样我们就可以直观地看到 Scaling 是如何影响 DiT 生成样本的质量的。事实上，缩放模型大小和 token 的数量可以显著地改善结果的视觉质量。

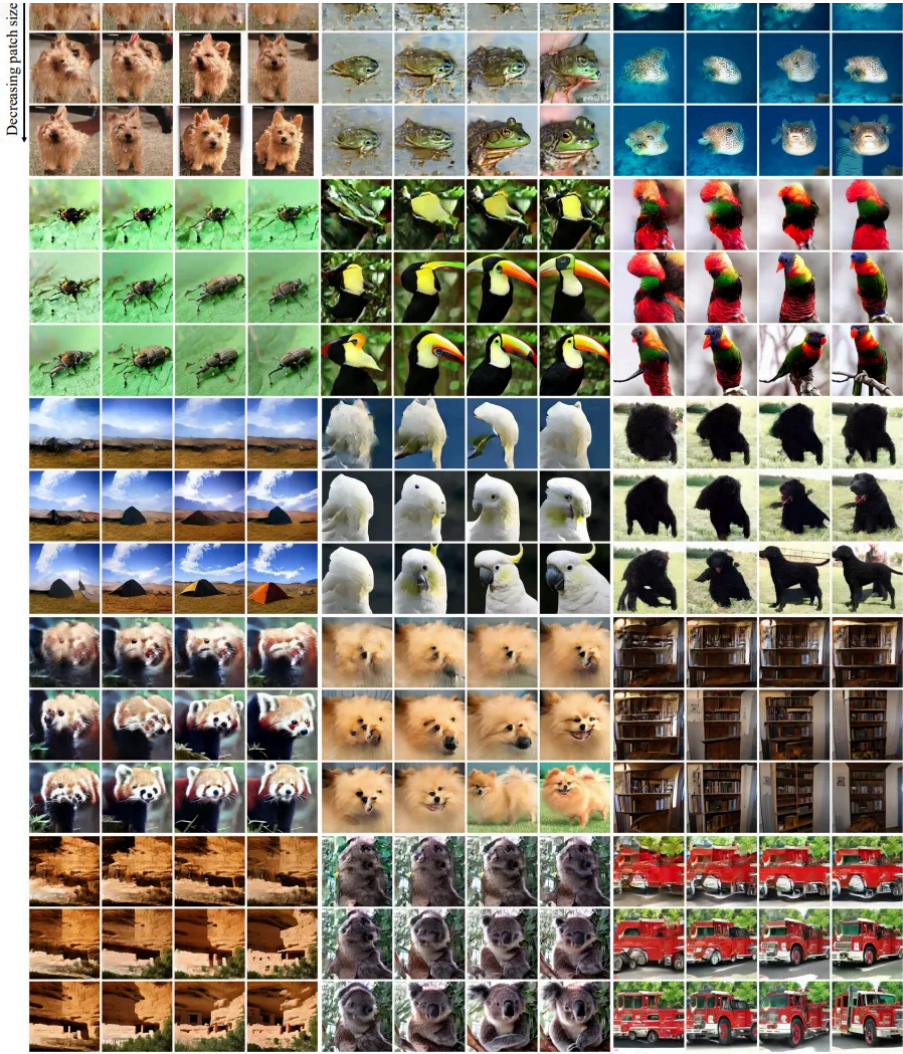


图8：缩放对于视觉质量的影响

1.6 DiT 实验结果

作者将 DiT 与最先进的生成模型进行了比较，结果如图9所示。DiT-XL/2 优于所有先前的扩散模型，将 LDM 实现的先前最佳 FID-50K 降低到 2.27。图5右侧显示 DiT-XL/2 (118.6 GFLOPs) 相对于 LDM-4 (103.6 GFLOPs) 等Latent Space U-Net 模型的计算效率很高，并且比 Pixel Space U-Net 模型更高效，例如 ADM (1120 GFLOPs) 或 ADM-U (742 GFLOPs)。

Model	FID↓	sFID↓	IS↑	Precision↑	Recall↑
BigGAN-deep [2]	6.95	7.36	171.4	0.87	0.28
StyleGAN-XL [53]	2.30	4.02	265.12	0.78	0.53
ADM [9]	10.94	6.02	100.98	0.69	0.63
ADM-U	7.49	5.13	127.49	0.72	0.63
ADM-G	4.59	5.25	186.70	0.82	0.52
ADM-G, ADM-U	3.94	6.14	215.84	0.83	0.53
CDM [20]	4.88	-	158.71	-	-
LDM-8 [48]	15.51	-	79.03	0.65	0.63
LDM-8-G	7.76	-	209.52	0.84	0.35
LDM-4	10.56	-	103.49	0.71	0.62
LDM-4-G (cfg=1.25)	3.95	-	178.22	0.81	0.55
LDM-4-G (cfg=1.50)	3.60	-	247.67	0.87	0.48
DiT-XL/2	9.62	6.85	121.50	0.67	0.67
DiT-XL/2-G (cfg=1.25)	3.22	5.28	201.77	0.76	0.62
DiT-XL/2-G (cfg=1.50)	2.27	4.60	278.24	0.83	0.57

图9: ImageNet 256×256 图像生成结果

作者在 ImageNet 上训练了一个新的 DiT-XL/2，这次分辨率是 512×512，3M training iterations，超参数与 256×256 模型相同。这个模型 latent 的维度是 64×64×4，然后 Patch Size 为 2，这样 Transformer 模型需要处理的 token 的数量就是 1024。如下图10所示是比较结果。DiT-XL/2 在此分辨率下再次优于所有先前的扩散模型，将 ADM 实现的先前最佳 FID 提高了 3.85 到 3.04。即使 token 的数量增加了，DiT-XL/2 的计算效率依然很高，比如 ADM 使用 1983 GFLOPs，ADM-U 使用 2813 GFLOPs，DiT-XL/2 仅仅使用 524.6 GFLOPs。

Class-Conditional ImageNet 512×512					
Model	FID↓	sFID↓	IS↑	Precision↑	Recall↑
BigGAN-deep [2]	8.43	8.13	177.90	0.88	0.29
StyleGAN-XL [53]	2.41	4.06	267.75	0.77	0.52
ADM [9]	23.24	10.19	58.06	0.73	0.60
ADM-U	9.96	5.62	121.78	0.75	0.64
ADM-G	7.72	6.57	172.71	0.87	0.42
ADM-G, ADM-U	3.85	5.86	221.72	0.84	0.53
DiT-XL/2	12.03	7.12	105.25	0.75	0.64
DiT-XL/2-G (cfg=1.25)	4.64	5.77	174.77	0.81	0.57
DiT-XL/2-G (cfg=1.50)	3.04	5.02	240.82	0.84	0.54

图10: ImageNet 512×512 图像生成结果

缩放模型大小还是采样次数？

Diffusion Model 的一个独特之处是它们可以通过在生成图像时增加采样步骤的数量来在训练期间使用额外的计算。也就是扩散模型的计算量既可以来自**模型本身的缩放**，也可以来自**采样次数的增加**。因此，作者在这里研究了通过使用更多的采样计算，较小的 DiT 模型是否可以胜过更大的模型。

作者计算了所有的 12 个 DiT 模型在 400K training iteration 时候的 FID 值，每张图分别使用 [16, 32, 64, 128, 256, 1000] sampling steps。

实验结果如下图11所示，考虑使用 1000 个采样步骤的 DiT-L/2 和使用 128 步的 DiT-XL/2。在这种情况下：

- DiT-L/2 使用 80.7 TFLOPs 对每张图像进行采样。
- DiT-XL/2 使用 15.2 TFLOPs 对每张图像进行采样。

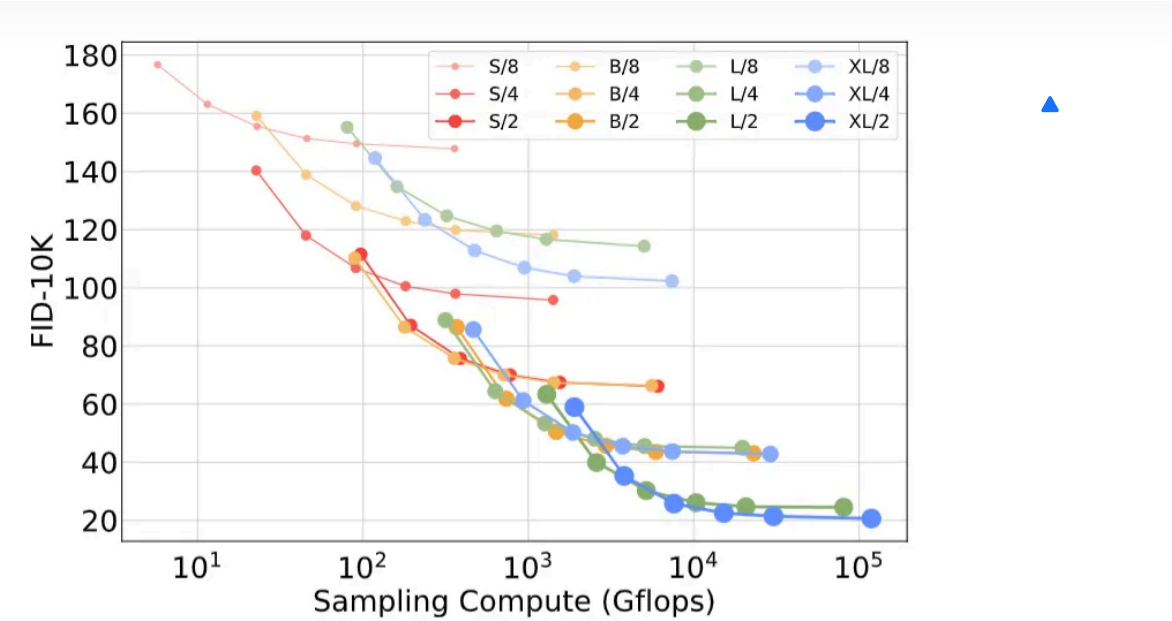


图11: 增加采样的计算量也无法弥补模型本身计算量的缺失

参考

1. [^] ^a ^b High-Resolution Image Synthesis with Latent Diffusion Models
2. [^] Language Models are Few-Shot Learners
3. [^] Generative Pretraining From Pixels
4. [^] Zero-Shot Text-to-Image Generation
5. [^] Language Models are Unsupervised Multitask Learners
6. [^] Hierarchical Text-Conditional Image Generation with CLIP Latents
7. [^] Diffusion Models Beat GANs on Image Synthesis
8. [^] ^a ^b Denoising Diffusion Probabilistic Models.
9. [^] PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications
10. [^] Conditional Image Generation with PixelCNN Decoders
11. [^] U-Net: Convolutional Networks for Biomedical Image Segmentation
12. [^] Diffusion Models Beat GANs on Image Synthesis
13. [^] FiLM: Visual Reasoning with a General Conditioning Layer
14. [^] ^a ^b <https://arxiv.org/pdf/2208.11970.pdf>
15. [^] A Style-Based Generator Architecture for Generative Adversarial Networks
16. [^] Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour

编辑于 2024-03-15 21:52 · IP 属地中国香港

扩散 Transformer

写下你的评论...

2 条评论

默认 最新

露白

我想问一下，对坐标用扩散模型，需要对输入的坐标进行归一化吗？

03-15 · IP 属地重庆

回复 喜欢

愚者

最好归一化（可以norm到[-1,1]或者[0,1]）。如果x0没有归一化，数值可能会不稳定。很多clip操作就会不合理。xt 由x0和epsilon加权得到，如果x0没有归一化，那么他和epsilon的尺度就不同，这可能也会是一个问题。

03-25 · IP 属地上海

回复 喜欢



AIGC 日积累
扩散模型和经典生成复原任务的模型介绍



推荐阅读



基于DiT模型的详解

鳄鱼家的帅气猪



FRP指定访问者:安全地暴露内网服务

刘遑 发表于Linux...



基于 gRPC 和 Sidecar 代理

Jimmy...